GPU Accelerated Graph Analytics

Welcome to Nvidia's workshop on Fundamentals of Accelerated Data Science.

This workshop serves as an introduction to data science with an emphasis on speed and efficiency.

It's been designed for new and practicing data scientists that are looking to accelerate workflows and optimize results with NVIDIA GPUs.

Let's get started. Here's an overview of the workshop.

Our goal is to guide you through the key stages of the data science workflow, helping you transform raw data into actionable insights.

These stages are broadly categorized into data preparation, model, training, inference, and deployment.

The learning objectives for this workshop include understanding the fundamental concepts and components of data science, exploring practical examples of accelerated data science workflows, and examining the tools and methods used to achieve acceleration in data science and discussing their broader implications.

To help you achieve these objectives, we've organized the workshop into three sections. Section 1 covers an introduction to data science and data manipulation, Section 2 focuses on graph analytics, and Section 3 covers a walkthrough of various machine learning tasks.

We'll conclude with a coding assessment which will allow you to apply what you've learned and earn a certificate for completing the workshop.

Let's begin with Section 2.

We'll start with a brief introduction on graph and graph data.

We'll then discuss some popular graph algorithms and use cases, which you'll get to apply in the Hands on Lab. We'll come with this topic from a conceptual as well as technical perspective to help you appreciate how graph analytics can be GPU accelerated.

A graph is a mathematical structure used to represent relationships between objects. It consists of two fundamental elements. A node or vertex is the fundamental unit of a graph representing an object or entity. An edge is the connection between two vertices showing a relationship or link. Nodes, edges, and graphs can be distinguished by their unique properties.

An undirected graph consists of nodes connected by edges without any specific direction. Some examples include relationships between people or transportation systems.

A directed graph or digraph consists of nodes connected by edges with specific directions. Some examples include web page lengths or project dependencies.

A weighted graph have edges with associated values or weights.

There are a few fundamental concepts and graph analytics that are crucial for understanding the structure and relationships within graphs. They form the basis for many advanced graph algorithms. These are degree and neighborhood. Degree helps us measure node connectivity.

The degree of a node is a simple yet powerful measure of its importance and connectivity within a graph.

In a directed graph, we distinguish between in degree, which is the number of edges pointing to the node, and out degree, which is the number of edges pointing away from the node.

This can be used for identifying influential nodes and social networks, analyzing connectivity and transportation systems, and detecting anomalies and network traffic.

Relatedly, the neighborhood helps us explore local structure. The neighborhood of a node represents its immediate connection and local environment within the graph. This can be used for analyzing information spread in networks.

By leveraging graph theory and graph based analytics, researchers and practitioners and various fields of research can uncover complex relationships, optimize processes, and gain deeper insights into the systems they study.

The versatility of graphs and representing and analyzing interconnected data makes them a powerful tool across diverse scientific and practical domains. Here we're showing a few examples.

- Molecules can be represented as graphs, with atoms as nodes and chemical bonds as edges. This representation allows for efficient analysis of molecular properties and structure activity relationships.
- The financial system can be modelled and studied as a graph, helping us evaluate robustness, resilience, and potential contagion effects, and graph is also used to study particle physics.
- Graphs play a crucial role in both social networks and recommender systems.

- In social networks, graphs are used to model the intricate web of connections between users. Nodes represent users, groups, or organizations. Edges represent relationships or interactions between the nodes. The resulting structure is often referred to as a social graph. We can map relationships between users, visualize communities, and analyze information flow and influence. This lets us create a personalized user experience, document user activities and connections, and create targeted advertising and content recommendations.
- A knowledge graph is a powerful data structure that represents a network of real world entities and illustrate the relationships between them. Nodes can represent objects, events, situations, or concepts. Edges are used to define and label the relationships between the nodes. This is a flexible way to model a complex and evolving data structure, especially with new information.

This structure of information is semantically rich, capturing important contacts within the data. By representing information as an interconnected network, knowledge graphs offer a powerful tool for organizing, analyzing, and deriving insights from complex data structure across various domains.

This can be used by LLM based chat bots when they need to be augmented with contextually rich information. Let's look at some ways to represent or visualize graph data. Here we're looking at the adjacency matrix across the rows and columns.

We have the unique node IDs and a binary indicator for the edges or connections. You'll notice that the adjacency matrix is symmetric for undirected graphs. We can also use other values to represent edge properties for weighted graphs.

Here we're looking at the degree matrix that summarizes the number of connections each unique node has. When we combine the two, we have the Laplacian matrix. This matrix is rich with information.

Someone with proficiency and tabular data manipulation can probably perform a lot of analytics with it. However, this isn't always the most efficient way to represent data.

This is observed by the high number of zeros, and it typically gets worse as the graph grows. The same data can be represented by the adjacency list. This representation contains as much information as the previous approach, but it's a lot more efficient.

We're also showing two other lists, one representing the list of node properties ordered by node ID, and a list representing the weights of each edge.

Though adjacency lists are widely used, the choice of how a graph is represented depends on graph density, the types of operation to be performed, memory constraints, and performance requirements.

Once we've established this data in a structured way, we can start to write algorithms to conduct the types of analytics we would like. One such example is the single source shortest path algorithm.

This is used to compute the shortest path from a designated source node to all reachable nodes in a weighted graph.

Given a graph and a source node ID, the algorithm calculates the distance to all reachable nodes. In this example, you see that node zero can reach all other nodes. It has a zero distance to itself.

Notably, there are two paths it can use to reach node 3, but the shortest path with a distance of three is returned, which is through node 2, as opposed to having to travel a distance of four through node one.

Given a graph and a source node ID, the algorithm calculates the distance to all reachable nodes. This is commonly used for Rd. networks, logistics, communications, and network analysis.

Centrality is the fundamental concept in graph theory and network analysis that measures the importance or influence of nodes within a graph.

It helps to identify key nodes based on their position and connections in the network. There's several types of centrality measures, each providing a different perspective on node importance. It might make more sense in the context of a social network and influence.

- Degree centrality measures importance based on the number of connections a node has. Higher degree indicates greater local influence.
- Closeness centrality measures how close a node is to all other nodes in the graph. Higher closeness indicates better position nodes for quick information spread.
- You might interpret this as the average length of the shortest path between a node and all other nodes between. This centrality quantifies how often a node acts as the bridge along the shortest paths between other nodes, higher between.
- This suggests greater importance and connecting different parts of the graph page. Rank centrality assigns importance to nodes based on the importance of their neighbors. It's useful for identifying influential nodes in the overall network structure.

Network X is a popular library that provides these functionalities for working with complex networks and graph structures. It can be accelerated with a GPU Python library KU Graph. Depending on the algorithm and graph size, speed UPS can range from 10 times to 500 times compared to CPU based Network X.

The key mechanism for the speed up comes from the ability to use GPU cores for parallel processing. NX Ku Graph allows data scientists to benefit from GPU acceleration in existing Network X workflows without modifying their code.

If you're starting with a Ku Graph data frame, you can efficiently convert it to a graph object and start analyzing. There are three ways to utilize NX Ku Graph.

- You can set the NX Ku Graph AUTO CONFIG environment variable when executing the Network X code, which will cause Network X to dispatch supported function to the GPU back end.
- You can see the performance gain is significant depending on several factors we discussed. You can also use the back end argument within a network access supported APIs.
- You can also explicitly create the NX Krew graph object which will have the APIs available for supported functions. This approach ensures specific behavior without potential runtime conversions, but of course requires the NX Krew Graph back end to be installed.

Go ahead and go through the hands on lab.